

Data Structures Assessment

Student Name: PAVITHRA R

Register No: 192421369

Course Code / Tag: CO2 AT1

Part 1: Infix to Postfix Conversion & Postfix Evaluation

1. Infix to Postfix Conversion

Given Infix Expression:

$B - (C / D + (E \% F * G) / H) * J$

Operator Precedence Rules Applied:

- Parentheses () — Highest precedence
- Modulo %, Multiplication *, Division / — Left-to-right association
- Addition +, Subtraction - — Lowest precedence

Step-by-Step Operator Stack Simulation:

TOKEN	OPERATOR STACK	OUTPUT POSTFIX EXPRESSION
B	Empty	B
-	-	B
(	- (	B
C	- (	B C
/	- (/	B C
D	- (/	B C D
+	- (+	B C D /
(	- (+ (	B C D /
E	- (+ (	B C D / E
%	- (+ (%	B C D / E
F	- (+ (%	B C D / E F
*	- (+ (*	B C D / E F %
G	- (+ (*	B C D / E F % G

TOKEN	OPERATOR STACK	OUTPUT POSTFIX EXPRESSION
)	- (+	B C D / E F % G *
/	- (+ /	B C D / E F % G *
H	- (+ /	B C D / E F % G * H
)	-	B C D / E F % G * H / +
*	- *	B C D / E F % G * H / +
J	- *	B C D / E F % G * H / + J
End	Empty	B C D / E F % G * H / + J * -

Final Postfix Expression: B C D / E F % G * H / + J * -

2. Postfix Evaluation

a) Expression: 9 3 8 4 / * +

STEP	TOKEN	OPERATION / CALCULATION	OPERAND STACK
1	9	Push 9	[9]
2	3	Push 3	[9, 3]
3	8	Push 8	[9, 3, 8]
4	4	Push 4	[9, 3, 8, 4]
5	/	Pop 4 and 8: $8 / 4 = 2$	[9, 3, 2]
6	*	Pop 2 and 3: $3 \times 2 = 6$	[9, 6]
7	+	Pop 6 and 9: $9 + 6 = 15$	[15]

Evaluation Result (a): 15

b) Expression: 6 2 + 6 3

Step-by-step processing of the provided sequence:

- Push 6, Push 2 $\rightarrow$ Stack: [6, 2]
- Encounter +: Pop 2 and 6 $\rightarrow 6 + 2 = 8$. Push 8 $\rightarrow$ Stack: [8]
- Push 6 $\rightarrow$ Stack: [8, 6]

• Push 3 $\rightarrow$ Stack: [8, 6, 3]

Evaluation State (b): Remaining Operand Stack contains [8, 6, 3] (Expression is incomplete as written in problem statement).

Part 2: Queue Operations Simulation

Queue Configuration: Linear Queue of Maximum Size = 5 (Index 0 to 4).

Initial Setup: FRONT = -1, REAR = -1, Queue Array = [_, _, _, _, _]

OPERATION	ACTION & STATUS	QUEUE ARRAY [0..4]	FRONT	REAR
Enqueue (21)	Inserted 21 at index 0	[21, _, _, _, _]	0	0
Enqueue (67)	Inserted 67 at index 1	[21, 67, _, _, _]	0	1
Enqueue (89)	Inserted 89 at index 2	[21, 67, 89, _, _]	0	2
Dequeue ()	Removed element 21	[_, 67, 89, _, _]	1	2
Enqueue (15)	Inserted 15 at index 3	[_, 67, 89, 15, _]	1	3
Enqueue (30)	Inserted 30 at index 4	[_, 67, 89, 15, 30]	1	4
Enqueue (52)	QUEUE OVERFLOW (REAR = Size - 1)	[_, 67, 89, 15, 30]	1	4
Dequeue ()	Removed element 67	[_, _, 89, 15, 30]	2	4
Dequeue ()	Removed element 89	[_, _, _, 15, 30]	3	4
Dequeue ()	Removed element 15	[_, _, _, _, 30]	4	4
Dequeue ()	Removed element 30 (Queue becomes empty)	[_, _, _, _, _]	-1	-1

Key Observations:

- Queue Overflow Condition:** In a standard linear queue, Enqueue (52) fails because REAR has reached index 4 (Max Size - 1), despite available empty slot at index 0 from previous Dequeue operation.
- Reset on Empty:** When the final element 30 is dequeued, both FRONT and REAR pointers reset to -1.